



**Department of Computer Science and Engineering
University of Puerto Rico
Mayagüez Campus**

INSO 6006 – Database Systems Engineering Fall 2026

**Project #1: Part Records and Buffer Replacement Policies
Due Date: October 2, 2026, 11:59 PM**

Objectives

- Understand fixed-size records, serialized blocks, CSV input, and buffer replacement policies.
- Understand how a buffer manager delegates page eviction to a replacement-policy interface.
- Gain experience with C++ standard-library containers, byte arrays, and systems programming practices.

Overview

You will complete several modules in a small database engine. The supplied project contains the buffer pool, buffer manager, block-file layer, replacement-policy interface, tests, and build configuration. You will receive the project directory without the implementation files listed below. Your task is to create those files and implement their declared interfaces. Do not modify the provided header files or the interfaces of supplied components.

The engine stores records in 4096-byte blocks. A buffer manager keeps blocks in memory and delegates eviction choices to a replacement policy. Your implementations must cooperate with the existing code and pass the supplied tests.

Part

Part.h is provided and must not be modified. It declares the Part structure and the PartColor enumeration. A Part has an integer identifier, a name of up to nine characters plus a terminating null character, a floating-point weight, a color value, a floating-point

price, and a material of up to nine characters plus a terminating null character.

A valid Part has a positive `part_id`, nonnegative `part_weight` and `part_price`, and a `part_color` in the inclusive range 0 through 5. The value zero for `part_id` is reserved as the free-slot marker.

PartSerializer

Implement `PartSerializer.cpp`. The disk layout is exactly 36 bytes, written field by field in this order:

`part_id` (4 bytes), `part_name` (10 bytes), `part_weight` (4 bytes),
`part_color` (4 bytes), `part_price` (4 bytes), `part_material` (10 bytes)

Do not write the in-memory Part structure with one copy operation because compiler padding is not part of the disk format. Implement serialization and deserialization, block initialization, complete block serialization and deserialization, and the slot-level helpers declared in `PartSerializer.h`.

A 4096-byte block holds 113 part records. A zero-initialized block is empty. Block readers stop at the first slot whose `part_id` is zero; writers must reject a Part whose identifier is zero.

PartCsv

Implement `PartCsv.cpp`. Each nonblank row has exactly six comma-separated fields in this order:

`part_id`, `part_name`, `part_weight`, `part_color`, `part_price`, `part_material`

Parse integer and floating-point values strictly: the entire input field must be consumed, and invalid, overflowing, or out-of-range values must be rejected. Invalid rows are not fatal. Print a diagnostic in the form "line N: reason", increment the skipped count, and continue with later rows. Valid rows are returned in file order. An input file that cannot be opened must report a diagnostic and return no valid parts.

PartGenerator

Implement `PartGenerator.cpp`. The generator returns the requested number of valid synthetic Part records. Identifiers must be sequential beginning with the supplied first identifier. Choose deterministic names, weights, colors, prices, and materials so the same arguments always produce the same records. Keep all generated text within the fixed field sizes and all generated values valid.

Replacement Policies

Implement `FIFOPolicy.cpp`, `LFUPolicy.cpp`, `LRUv2Policy.cpp`, and `MRUPolicy.cpp`. Each class implements the supplied `ReplacementPolicy` interface. The buffer manager calls `init` once, calls `on_load` for a newly resident page, `on_access` for a hit, and `on_remove` before a page leaves a frame. `pick_victim` receives only unpinned candidate frames and must return one of those frames, or no value when no candidate can be selected.

FIFO: Evict the candidate that entered the pool earliest. A later access does not change FIFO order.

LRUv2: Evict the least recently used candidate. Maintain most recently used at one end of the order and least recently used at the other. Use the list and hash-map design suggested by the header.

MRU: Evict the most recently used candidate. Its tracking order is the same as LRU, but victim selection is from the opposite end.

LFU: Evict the candidate with the fewest accesses while resident. Break equal-frequency ties by least-recent access. A newly loaded page starts with frequency one, and counters reset when a page is removed.

Your policy bookkeeping must remain consistent when frames are loaded, accessed, removed, and reused. Do not examine the buffer pool directly; the policy only reasons about frame identifiers and the candidates supplied to it.

Tasks for Project 1

You will receive the Git repository with the following implementation files removed. Create and implement these files:

```
part/PartSerializer.cpp
part/PartCsv.cpp
part/PartGenerator.cpp
policy/FIFOPolicy.cpp
policy/LFUPolicy.cpp
policy/LRUv2Policy.cpp
policy/MRUPolicy.cpp
```

All header files, `ReplacementPolicy.h`, the buffer manager and buffer pool, block-file code, person-record code, configuration code, test programs, and build configuration are provided. Do not modify `Part.h`.

Testing and Submission

Build the project with CMake and run the supplied check executable or CTest suite. Test normal cases, boundary slots, invalid CSV rows, empty blocks, policy tie breaking,

pinned-frame exclusion, and frame reuse. Submit only the requested implementation files unless your instructor specifies a different submission procedure.

PROJECT DUE DATE: OCTOBER 2, 2026, 11:59 PM